

Optimizing Cache Usage in Scientific and Machine Learning Workloads

Introduction

Large-scale computations in scientific disciplines, including Finite Element Methods (FEM), Computational Fluid Dynamics (CFD), alongside neural network training, encounter a significant performance obstacle termed the "Memory Wall." Although the processing capabilities of arithmetics have been greatly improved, the improvements in memory throughput and response times have not improved substantially. This research report explores how contemporary architecture can address these constraints through cache optimizations implemented at the software level. The emphasis lies on identifying origins of memory inefficiency, particularly within data-parallel computational kernels, while suggesting algorithmic modifications such as loop restructuring, memory layout adjustments, and polyhedral model refinements to boost both spatial and temporal data reuse.

Background / Related Work

Current high-performance computing (HPC) relies on the complex memory hierarchy to provide data to processing units that are challenging. However, inconsistent access patterns often degrade cache performance. The disparity between computation speed and data transport is significant. Patterson and Hennessy (2017) describe this hierarchy, noting that performance is increasingly defined by the ability to keep active data within the L1/L2 caches.

The traditional object-oriented paradigm is called Array of Structures (AoS) and is a compilation of the attributes of a particle (e.g. [x, y, z, mass]) but in the case of computational kernels making use of only a subset of these attributes as in, e.g., positional coordinates, AoS requires unnecessary usage of cache-line capacity of idle data (Strzodka, 2012). Conversely, the Structure of Arrays (SoA) layout places the attributes in adjacent arrays and, therefore, maximizes the utilisation of the currently fetched cache line and supports Single Instruction, Multiple Data (SIMD) vectorisation (Intel, 2023). The Polyhedral Models such as Pluto and Polly, represent loop nests as a mathematical polytope, and use affine transformations to optimize spatial and temporal locality and parallelism automatically, and in many cases perform better than manual tuning of complex stencil computations (Bondhugula, et al., 2008).

Methodology / System Design

To address cache inefficiencies in data-parallel kernels, this analysis proposes a dual-strategy optimization: implementing Loop Tiling (Blocking) to enforce temporal locality and switching from Array of Structures (AoS) to Structure of Arrays (SoA) to maximize spatial locality.

The assumptions are that the target hardware is a modern multi-core CPU (for example, x86-64) equipped with a 32KB L1 data cache and 64-byte cache lines, the workload involves matrix or grid dimensions N that significantly exceed the capacity of the L2/L3 cache such that $N^2 \times 8$ bytes is significantly larger compared to the cache size, and data is stored in row-major order, which is the standard layout used in C and C++ programs.

This approach improves the performance of the cache by changing the data layouts into structure-of-arrays (SoA) instead of array-of-structures (AoS), so that every 64-byte cache line can carry eight double-precision values instead of one or two when a subset of fields is being accessed (Strzodka, 2012). The use of blocking based on the cache size which is tuned to L1 (i. e. 32 by 32) can result in over 90 per cent of L1 hits (Lam et al., 1991). This decrease in access frequency of DRAM is important since the DRAM latency (~60-100ns) is orders of magnitude slower than L1 cache latency (~1ns).

Feature	Baseline System (Naive)	Optimized System (Tiled + SoA)	Architectural Impact
Memory Access	Strided access (e.g., stride-4 or stride-N)	Unit-stride (sequential) access	Eliminates cache line pollution; enables hardware prefetchers to predict next lines effectively.
Data Layout	Array of Structures (AoS): [x,y,z]	Structure of Arrays (SoA): [x...][y...]	SoA enables SIMD vector units to use 100% of the cache line to load contiguous data (Intel, 2023).
Loop Structure	Deeply nested loops iterating over full arrays	Tiled loops iterating over sub-blocks (M x M)	Maintains the inner loop's working set below the L1/L2 cache size (Lam et al., 1991).
Reuse Distance	High	Low	Keeps valuable information from being removed before it may be utilized again.

Discussion

The suggested optimizations give radical increases in the throughput, and polyhedral compilation instruments often exhibit speedups of $2\times$ to $15\times$ on stencil computations (Bondhugula et al., 2008). However, an improvement in performance also comes at the expense of the code's complexity and loss of readability. Tiled loop nests are very difficult to debug and maintain than naive loops. Moreover, the calculation of an optimal tile size is a hardware-specific issue: code written to be optimal performance on a specific Intel CPU cache can perform poorly on AMD chips or ARM-based embedded devices, so the auto-Tuning systems must be used to maintain portability.

Effective computer architecture does not only limit itself to speed of execution but also to environmental sustainability. Movement of data forms the largest energy user in modern-day computing; retrieving a 64-bit operand out of DRAM requires around 1200-2000 (pJ), but the actual floating-point operation requires only about 4 pJ (Dally, 2023; Horowitz, 2014). The energy footprint of the workload is removed in line with the employed cache-aware optimizations that can reduce DRAM access by at least ten times. With a global data center and the large-scale adoption of edge AI, memory traffic optimization cannot be reduced in the list of alternatives to lessen carbon emissions and electric power usage. Moreover, thermal stress relief on the hardware components will help eliminate thermal throttling and physical degradation to increase the life of the devices and help in the decline of electronic waste (e-waste).

Conclusion

This study has demonstrated that memory access patterns are the primary determinant of system performance in data-parallel scientific applications. With the use of Structure-of-Arrays (SoA) layouts and Loop Tiling, system architectures and software developers can overcome successful bridging of the performance divided between computational performance and memory bandwidth. The methodology that follows does not only enhance the user experience by reducing the time of simulation but also contributes to sustainable computing by limiting the energy-inefficient data transportation. Future work needs to focus on how to incorporate these changes in domain-specific compilers thus automating the generation of energy-efficient code that is specific to heterogeneous architecture.